

Vertex AI Workbench：使用 BigQuery 資料訓練 TensorFlow 模型

來源：<https://codelabs.developers.google.com/vertex-workbench-intro?hl=zh-tw>

步驟數：7

目錄

1. [總覽](#)
2. [Vertex AI 簡介](#)
3. [用途總覽](#)
4. [設定環境](#)
5. [在 BigQuery 中探索資料集](#)
6. [在 TensorFlow 核心上訓練機器學習模型](#)
7. [清除所用資源](#)

步驟 1 · 約 1 分鐘

1. 總覽

在本實驗室中，您將瞭解如何使用 Vertex AI Workbench 進行資料探索和機器學習模型訓練。

課程內容

內容如下：

- 建立及設定 Vertex AI Workbench 執行個體
- 使用 Vertex AI Workbench BigQuery 連接器
- 在 Vertex AI Workbench 核心上訓練模型

在 Google Cloud 上執行這個實驗室的總費用約為 **\$1 美元**。

2. Vertex AI 簡介

本實驗室使用 Google Cloud 最新推出的 AI 產品服務。[Vertex AI](#) 整合了 Google Cloud 機器學習服務，提供流暢的開發體驗。以 AutoML 訓練的模型和自訂模型，先前需透過不同的服務存取。這項新服務將兩者併至單一 API，並加入其他新產品。您也可以將現有專案遷移至 Vertex AI。

Vertex AI 包含許多不同的產品，可支援端對端機器學習工作流程。本實驗室將著重於 Vertex AI Workbench。

Vertex AI Workbench 與資料服務 (例如 Dataproc、Dataflow、BigQuery 和 Dataplex) 和 Vertex AI 深度整合，可協助使用者快速建構端對端筆記本工作流程。資料科學家可透過這個平台連線至 GCP 資料服務、分析資料集、實驗各種建模技術、將訓練好的模型部署至正式環境，以及在模型生命週期中管理 MLOps。

3. 用途總覽

在本實驗室中，您將探索「London Bicycles Hire」(倫敦單車租借) 資料集。這項資料包含 2011 年至今，倫敦公共自行車共用計畫的自行車行程資訊。您將透過 Vertex AI Workbench BigQuery 連接器，在 BigQuery 中探索這個資料集。接著，您將使用 pandas 將資料載入 Jupyter Notebook，並訓練 TensorFlow 模型，根據行程發生時間和騎乘距離預測單車行程的時長。

本實驗室會使用 [Keras 預先處理層](#)，轉換及準備模型訓練的輸入資料。這個 API 可讓您直接在 TensorFlow 模型圖中建構預先處理程序，確保訓練資料和服務資料經過相同的轉換，降低訓練/服務偏差的風險。請注意，自 TensorFlow 2.6 起，這個 API 已穩定運作。如果您使用舊版 TensorFlow，請匯入 [實驗符號](#)。

步驟 4 · 約 10 分鐘

4. 設定環境

您必須擁有已啟用計費功能的 Google Cloud Platform 專案，才能執行這項程式碼研究室。如要建立專案，請按照[這裡的說明](#)操作。

步驟 1：啟用 Compute Engine API

前往「Compute Engine」，然後選取「啟用」（如果尚未啟用）。

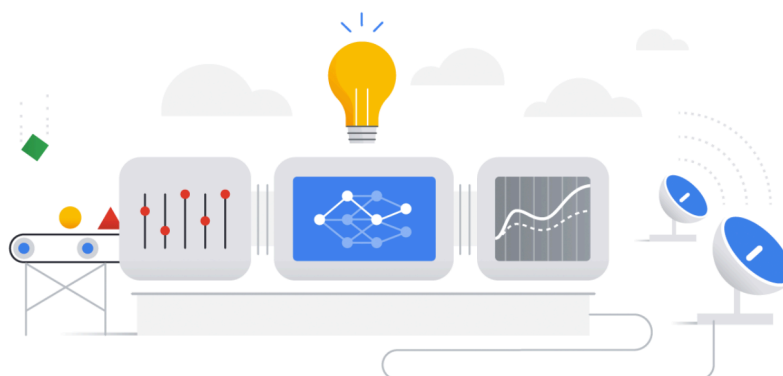
步驟 2：啟用 Vertex AI API

前往 [Cloud 控制台](#) 的 [Vertex AI 專區](#)，然後點選「啟用 Vertex AI API」。

Get started with Vertex AI

Vertex AI empowers machine learning developers, data scientists, and data engineers to take their projects from ideation to deployment, quickly and cost-effectively. [Learn more](#)

ENABLE VERTEX AI API



Region
us-central1 (Iowa)

Prepare your training data

Collect and prepare your data, then import it into a dataset to train a model

+ CREATE DATASET

Train your model

Train a best-in-class machine learning model with your dataset. Use Google's AutoML, or bring your own code.

+ TRAIN NEW MODEL

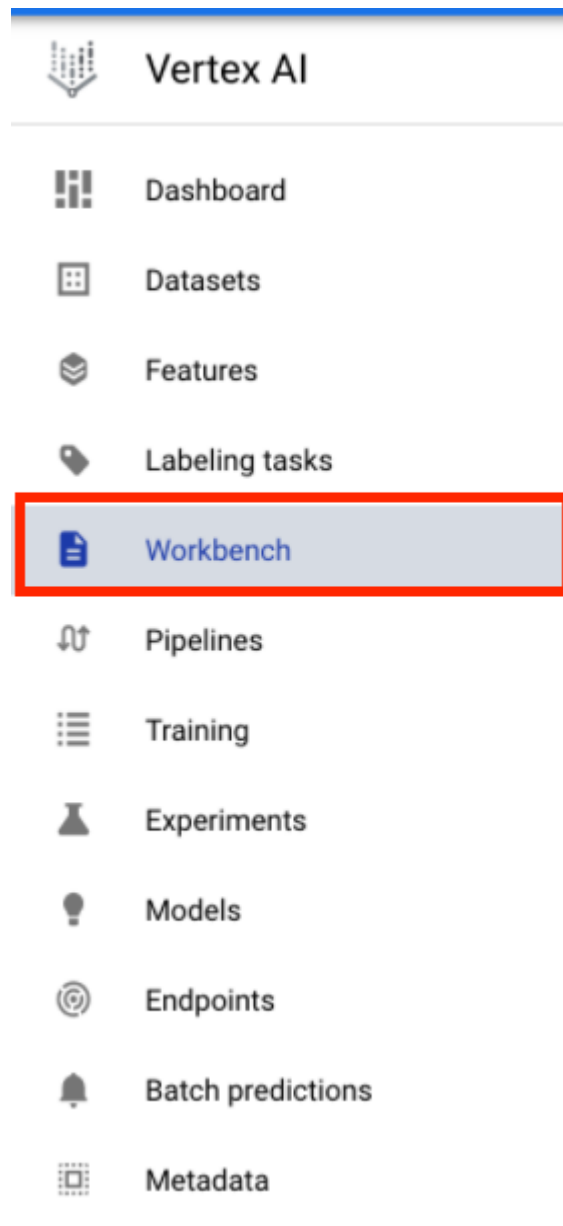
Get predictions

After you train a model, you can use it to get predictions, either online as an endpoint or through batch requests

+ CREATE BATCH PREDICTION

步驟 3：建立 Vertex AI Workbench 執行個體

在 Cloud 控制台的「Vertex AI」部分中，按一下「Workbench」：



如果尚未啟用 Notebooks API，請先啟用。



Notebooks API

Google Enterprise API

Notebooks API is used to manage notebook resources in Google Cloud.

ENABLE

TRY THIS API [↗](#)

OVERVIEW

PRICING

DOCUMENTATION

Overview

Notebooks API is used to manage notebook resources in Google Cloud.

Additional details

Type: [SaaS & APIs](#)

Last updated: 15/09/2021

Category: [Google Enterprise APIs](#)

Service name: notebooks.googleapis.com

啟用後，按一下「MANAGED NOTEBOOKS」(代管型筆記本)：

Workbench

[+ NEW NOTEBOOK](#)

[REFRESH](#)

MANAGED NOTEBOOKS

USER-MANAGED NOTEBOOKS

Managed notebooks provide JupyterLab services and flexible comp

然後選取「新增記事本」。

[+ NEW NOTEBOOK](#)

[REFRESH](#)

▶ ST/

BOOKS

PREVIEW

USER-MANAGED NOTEBOOKS

ooks service has been moved under the Vertex AI Workbenc

為筆記本命名，然後在「權限」下方選取「服務帳戶」

Create a managed notebook

Notebook name *

london-bikes-codelab

63-character limit with lowercase letters, digits or '-' only. Must start with a letter. Cannot end with a '-'.

Region

us-central1 (Iowa)



Permission

JupyterLab access modes determine who can use a notebook instance and which credentials are used to call Google APIs. **This cannot be changed once the notebook is created.**

☒ Service account

Service account mode allows anyone who is granted the `iam.serviceAccounts.actAs` permission on the specified service account to access the JupyterLab UI. [Learn more](#)

☐ Single user only

Single-user-only mode restricts access to the user specified below. [Learn more](#)

☒ Use Compute Engine default service account



If you have made changes to the Compute Engine default service account, make sure it still has sufficient API permissions. [Learn more](#)

選取「進階設定」。

在「安全性」下方，選取「啟用終端機」(如果尚未啟用)。

Security

- ☒ Enable nbconvert 
- ☒ Enable file downloading from Notebook UI
- ☒ Enable terminal 

其他進階設定可以保留原樣。

接著點選「建立」。

建立執行個體後，請選取「OPEN JUPYTERLAB」。

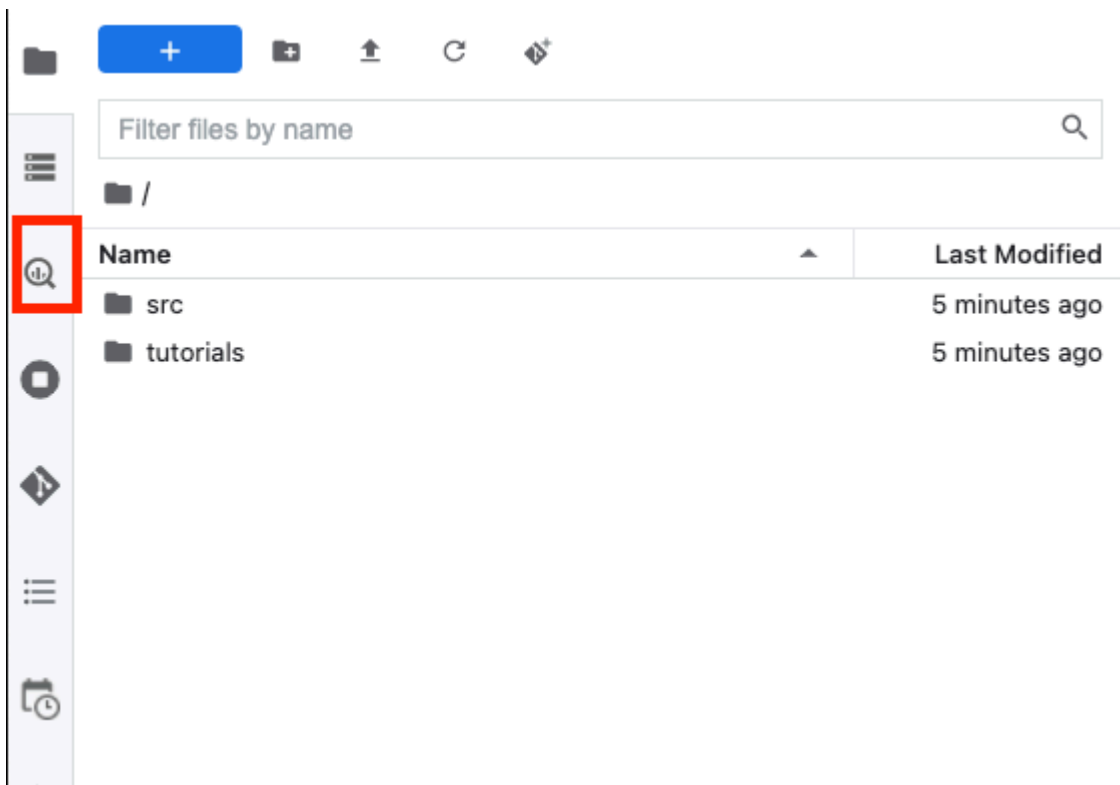
london-bikes-codelab

OPEN JUPYTERLAB

步驟 5 · 約 10 分鐘

5. 在 BigQuery 中探索資料集

在 Vertex AI Workbench 執行個體中，前往左側並點選「BigQuery in Notebooks」連接器。



透過 BigQuery 連接器，您可以輕鬆探索及查詢 BigQuery 資料集。除了專案中的資料集，您也可以按一下「新增專案」按鈕，探索其他專案中的資料集。

BigQuery

[Open SQL editor](#)

Resources



在本實驗室中，您將使用 BigQuery 公開資料集的資料。向下捲動，直到找到「london_bicycles」 **london_bicycles** 資料集。您會看到這個資料集有兩個資料表，分別是 **cycle_hire** 和 **cycle_stations**。以下將逐一說明。



london_bike_ds

首先，請按兩下 **cycle_hire** 資料表。您會看到資料表在新分頁中開啟，並顯示資料表結構定義和中繼資料，例如資料列數和大小。

cycle_hire

cycle_hire

<> Query table

Generate Statistics

Details

Preview

Description

Labels

None

Table info

Table ID	bigquery-public-data.london_bicycles.cycle_hire
Table size	2.59 GB
Number of rows	24,369,201
Created	Jul 11, 2017, 7:32:39 AM
Table expiration	Never
Last modified	May 8, 2019, 4:21:52 AM
Data location	EU

Schema

Field name	Type	Mode	Description
rental_id	INTEGER	REQUIRED	
duration	INTEGER	NULLABLE	Duration of the bike trip in seconds.
bike_id	INTEGER	NULLABLE	
end_date	TIMESTAMP	NULLABLE	
end_station_id	INTEGER	NULLABLE	
end_station_name	STRING	NULLABLE	

按一下「預覽」分頁標籤，即可查看資料樣本。我們來執行簡單的查詢，看看熱門的旅程有哪些。首先，按一下「查詢表格」按鈕。

Launcher

cycle_hire

cycle_hire

<> Query table

Generate Statistics

Details

Preview

Row	rental_id	duration	bike_id	end_date	end_station_id	end_station_name	start_date	start_station_id	start_station_name
1	59315896	2040	5640	1.47620454E9	303	Albert Gate, Hyde Park	1.4762025E9	303	Albert Gate, Hyde Park
2	55528181	4440	9149	1.46763816E9	303	Albert Gate, Hyde Park	1.46763372E9	281	Smith Square, Westminster
3	46380998	9600	12263	1.43854176E9	303	Albert Gate, Hyde Park	1.43853216E9	303	Albert Gate, Hyde Park
4	41927064	2160	1552	1.42668576E9	303	Albert Gate, Hyde Park	1.4266836E9	303	Albert Gate, Hyde Park
5	65576237	2400	700	1.4962455E9	303	Albert Gate, Hyde Park	1.4962431E9	404	Palace Gate, Kensington Gardens
6	66047428	7320	11490	1.49738076E9	303	Albert Gate, Hyde Park	1.49737344E9	166	Seville Street, Knightsbridge
7	52979425	10380	8997	1.46092296E9	303	Albert Gate, Hyde Park	1.46091258E9	303	Albert Gate, Hyde Park
8	57450651	2160	4604	1.4717133E9	303	Albert Gate, Hyde Park	1.47171114E9	304	Cumberland Gate, Hyde Park
9	65819982	2820	7728	1.49684658E9	303	Albert Gate, Hyde Park	1.49684376E9	151	Chepstow Villas, Notting Hill
10	48512262	2100	6329	1.4441484E9	303	Albert Gate, Hyde Park	1.4441463E9	573	Limerston Street, West Chelsea
11	54727828	2700	10909	1.46550816E9	303	Albert Gate, Hyde Park	1.46550546E9	307	Black Lion Gate, Kensington Gardens
12	46321627	8580	10168	1.43844174E9	303	Albert Gate, Hyde Park	1.43843316E9	292	Montpelier Street, Knightsbridge
13	59940087	2640	9506	1.47792732E9	303	Albert Gate, Hyde Park	1.47792468E9	303	Albert Gate, Hyde Park
14	56088744	4620	10646	1.4688717E9	303	Albert Gate, Hyde Park	1.46886708E9	303	Albert Gate, Hyde Park

然後將下列內容貼到 SQL 編輯器，並點選「提交查詢」。

```
SELECT
  start_station_name,
  end_station_name,
  IF(start_station_name = end_station_name,
    TRUE,
    FALSE) same_station,
  AVG(duration) AS avg_duration,
  COUNT(*) AS total_rides
FROM
  `bigquery-public-data.london_bicycles.cycle_hire`
GROUP BY
  start_station_name,
  end_station_name,
  same_station
ORDER BY
  total_rides DESC
```

查詢結果顯示，往返海德公園角站的單車行程最受歡迎。

Submit Query

```
1 SELECT
2   start_station_name,
3   end_station_name,
4   IF(start_station_name = end_station_name,
5     TRUE,
6     FALSE) same_station,
7   AVG(duration) AS avg_duration,
8   COUNT(*) AS total_rides
9 FROM
10  `bigquery-public-data.london_bicycles.cycle_hire`
11 GROUP BY
12  start_station_name,
```

Query results Query complete (1.5 GB processed)

Row	start_station_name	end_station_name	same_station	avg_duration	total_rides
1	Hyde Park Corner, Hyde Park	Hyde Park Corner, Hyde Park	true	3358.9778343949047	58875
2	Black Lion Gate, Kensington Gardens	Black Lion Gate, Kensington Gardens	true	3240.663383814922	32289
3	Albert Gate, Hyde Park	Albert Gate, Hyde Park	true	2870.7573084677424	31744
4	Aquatic Centre, Queen Elizabeth Olympic Park	Aquatic Centre, Queen Elizabeth Olympic Park	true	3515.696265149641	24258
5	Triangle Car Park, Hyde Park	Triangle Car Park, Hyde Park	true	2975.066555740434	21636
6	Speakers' Corner 1, Hyde Park	Speakers' Corner 1, Hyde Park	true	3608.825475063233	15419
7	Palace Gate, Kensington Gardens	Palace Gate, Kensington Gardens	true	2907.899761336515	15084
8	Speakers' Corner 2, Hyde Park	Speakers' Corner 2, Hyde Park	true	3412.4156829679587	14232

接著，按兩下「cycle_stations」**cycle_stations**資料表，即可查看各個車站的相關資訊。

我們要彙整 **cycle_hire** 和 **cycle_stations** 資料表。**cycle_stations** 表格包含每個車站的經緯度。您將使用這項資訊，計算起點和終點之間的距離，估算每次騎乘單車的距離。

如要進行這項計算，請使用 **BigQuery 地理位置函式**。具體來說，您會將每個經緯度字串轉換為 **ST_GEOPOINT**，並使用 **ST_DISTANCE** 函式計算兩點之間的直線距離 (以公尺為單位)。您會使用這個值做為每趟單車行程的行駛距離 Proxy。

將下列查詢複製到 SQL 編輯器，然後按一下「提交查詢」。請注意，JOIN 條件中有三個資料表，因為我們需要將 **stations** 資料表聯結兩次，才能取得單車的起點和終點站經緯度。

```

WITH staging AS (
  SELECT
    STRUCT(
      start_stn.name,
      ST_GEOGPOINT(start_stn.longitude, start_stn.latitude) AS POINT,
      start_stn.docks_count,
      start_stn.install_date
    ) AS starting,
    STRUCT(
      end_stn.name,
      ST_GEOGPOINT(end_stn.longitude, end_stn.latitude) AS point,
      end_stn.docks_count,
      end_stn.install_date
    ) AS ending,
    STRUCT(
      rental_id,
      bike_id,
      duration, --seconds
      ST_DISTANCE(
        ST_GEOGPOINT(start_stn.longitude, start_stn.latitude),
        ST_GEOGPOINT(end_stn.longitude, end_stn.latitude)
      ) AS distance, --meters
      start_date,
      end_date
    ) AS bike
  FROM `bigquery-public-data.london_bicycles.cycle_stations` AS start_stn
  LEFT JOIN `bigquery-public-data.london_bicycles.cycle_hire` as b
  ON start_stn.id = b.start_station_id
  LEFT JOIN `bigquery-public-data.london_bicycles.cycle_stations` AS end_stn
  ON end_stn.id = b.end_station_id
  LIMIT 700000)

SELECT * from STAGING

```

步驟 6 · 約 10 分鐘

6. 在 TensorFlow 核心上訓練機器學習模型

Vertex AI Workbench 具有運算相容性層，可讓您從單一筆記本執行個體啟動 TensorFlow、PySpark、R 等核心。在本實驗室中，您將使用 TensorFlow 核心建立筆記本。

建立 DataFrame

查詢執行完畢後，按一下「Copy code for DataFrame」(複製 DataFrame 的程式碼)。這樣一來，您就能將 Python 程式碼貼到連結至 BigQuery 用戶端的筆記本，並將資料擷取為 pandas DataFrame。

Submit Query

This query will process 1.3 GB when run.

```
1 WITH staging AS (  
2   SELECT  
3     STRUCT(  
4       start_stn.name,  
5       ST_GEOGPOINT(start_stn.longitude, start_stn.latitude) AS POINT,  
6       start_stn.docks_count,  
7       start_stn.install_date  
8     ) AS starting,  
9     STRUCT(  
10      end_stn.name,  
11      ST_GEOGPOINT(end_stn.longitude, end_stn.latitude) AS point,  
12      end_stn.docks_count
```

Query results Query complete (1.3 GB processed)

Copy code for DataFrame Explore in Data Studio

接著返回啟動器，建立 TensorFlow 2 筆記本。

Notebook

Python (Local)

PySpark (Local)

Pytorch (Local)

R (Local)

TensorFlow 2 (Local)

XGBoost (Local)

Console

Python (Local)

PySpark (Local)

Pytorch (Local)

R (Local)

TensorFlow 2 (Local)

XGBoost (Local)

Other

Terminal

Text File

Markdown File

Python File

R File

Show Contextual Help

在筆記本的第一個儲存格中，貼上從查詢編輯器複製的程式碼。畫面應如下所示：

```

# The following two lines are only necessary to run once.
# Comment out otherwise for speed-up.
from google.cloud.bigquery import Client, QueryJobConfig
client = Client()

query = """WITH staging AS (
    SELECT
        STRUCT(
            start_stn.name,
            ST_GEOGPOINT(start_stn.longitude, start_stn.latitude) AS POINT,
            start_stn.docks_count,
            start_stn.install_date
        ) AS starting,
        STRUCT(
            end_stn.name,
            ST_GEOGPOINT(end_stn.longitude, end_stn.latitude) AS point,
            end_stn.docks_count,
            end_stn.install_date
        ) AS ending,
        STRUCT(
            rental_id,
            bike_id,
            duration, --seconds
            ST_DISTANCE(
                ST_GEOGPOINT(start_stn.longitude, start_stn.latitude),
                ST_GEOGPOINT(end_stn.longitude, end_stn.latitude)
            ) AS distance, --meters
            start_date,
            end_date
        ) AS bike
    FROM `bigquery-public-data.london_bicycles.cycle_stations` AS start_stn
    LEFT JOIN `bigquery-public-data.london_bicycles.cycle_hire` as b
    ON start_stn.id = b.start_station_id
    LEFT JOIN `bigquery-public-data.london_bicycles.cycle_stations` AS end_stn
    ON end_stn.id = b.end_station_id
    LIMIT 700000)

SELECT * from STAGING"""
job = client.query(query)
df = job.to_dataframe()

```

為配合本實驗室的學習目標，我們將資料集限制為 700000，以縮短訓練時間。但您可以隨意修改查詢，並對整個資料集進行實驗。

接著，匯入必要的程式庫。

```

from datetime import datetime
import pandas as pd
import tensorflow as tf

```

執行下列程式碼，建立縮減的 DataFrame，其中只包含本練習機器學習部分所需的資料欄。


```
values = df['bike'].values
duration = list(map(lambda a: a['duration'], values))
distance = list(map(lambda a: a['distance'], values))
dates = list(map(lambda a: a['start_date'], values))
data = pd.DataFrame(data={'duration': duration, 'distance': distance, 'start_date': dates})
data = data.dropna()
```

`start_date` 欄是 Python `datetime`。您不會直接在模型中使用這項 `datetime`，而是要建立兩項新特徵，指出單車行程發生的星期幾和幾點。

```
data['weekday'] = data['start_date'].apply(lambda a: a.weekday())
data['hour'] = data['start_date'].apply(lambda a: a.time().hour)
data = data.drop(columns=['start_date'])
```

最後，將時間長度欄位從秒轉換為分鐘，方便理解

```
data['duration'] = data['duration'].apply(lambda x: float(x / 60))
```

檢查格式化 `DataFrame` 的前幾列。現在，每趟單車行程都會顯示行程發生的星期幾和時段，以及騎乘距離。您將根據這些資訊預測行程時間。

```
data.head()
```

```
[9]:
```

	duration	distance	weekday	hour
0	18.0	1975.389052	0	19
1	7.0	1190.357741	1	21
2	14.0	2196.101165	3	23
3	21.0	2333.228131	3	10
4	28.0	2540.177386	2	12

建立及訓練模型前，您需要將資料分割為訓練集和驗證集。

```
# Use 80/20 train/eval split
train_size = int(len(data) * .8)
print ("Train size: %d" % train_size)
print ("Evaluation size: %d" % (len(data) - train_size))

# Split data into train and test sets
train_data = data[:train_size]
val_data = data[train_size:]
```

建立 TensorFlow 模型

您將使用 [Keras Functional API](#) 建立 TensorFlow 模型。如要預先處理輸入資料，請使用 Keras 預先處理層 API。

下列公用函式會從 pandas Dataframe 建立 `tf.data.Dataset`。

```
def df_to_dataset(dataframe, label, shuffle=True, batch_size=32):
    dataframe = dataframe.copy()
    labels = dataframe.pop(label)
    ds = tf.data.Dataset.from_tensor_slices((dict(dataframe), labels))
    if shuffle:
        ds = ds.shuffle(buffer_size=len(dataframe))
    ds = ds.batch(batch_size)
    ds = ds.prefetch(batch_size)
    return ds
```

使用上述函式建立兩個 `tf.data.Dataset`，一個用於訓練，另一個用於驗證。您可能會看到一些警告，但可以放心忽略。

```
train_dataset = df_to_dataset(train_data, 'duration')
validation_dataset = df_to_dataset(val_data, 'duration')
```

您將在模型中使用下列前處理層：

- **正規化層**：對輸入特徵執行特徵層級的正規化。
- **IntegerLookup 層**：將整數類別值轉換為整數索引。
- **CategoryEncoding 層**：將整數類別特徵轉換為 one-hot、multi-hot 或 TF-IDF 密集表示法。

請注意，這些層級無法訓練。而是透過 `adapt()` 方法將預先處理層公開給訓練資料，藉此設定預先處理層的狀態。

下列函式會建立可對距離特徵使用的正規化層。您可以使用訓練資料的 `adapt()` 方法，在調整模型前設定狀態。這會計算平均值和變異數，以用於正規化。稍後將驗證資料集傳遞至模型時，系統會使用訓練資料計算出的相同平均值和變異數，來調整驗證資料。

```
def get_normalization_layer(name, dataset):
    # Create a Normalization layer for our feature.
    normalizer = tf.keras.layers.Normalization(axis=None)

    # Prepare a Dataset that only yields our feature.
    feature_ds = dataset.map(lambda x, y: x[name])

    # Learn the statistics of the data.
    normalizer.adapt(feature_ds)

    return normalizer
```

同樣地，下列函式會建立類別編碼，用於小時和星期幾特徵。

```
def get_category_encoding_layer(name, dataset, dtype, max_tokens=None):
    index = tf.keras.layers.IntegerLookup(max_tokens=max_tokens)

    # Prepare a Dataset that only yields our feature
    feature_ds = dataset.map(lambda x, y: x[name])

    # Learn the set of possible values and assign them a fixed integer index.
    index.adapt(feature_ds)

    # Create a Discretization for our integer indices.
    encoder = tf.keras.layers.CategoryEncoding(num_tokens=index.vocabulary_size())

    # Apply one-hot encoding to our indices. The lambda function captures the
    # layer so we can use them, or include them in the functional model later.
    return lambda feature: encoder(index(feature))
```

接著，請建立模型的預先處理部分。首先，請為每個特徵建立 `tf.keras.Input` 層。

```
# Create a Keras input layer for each feature
numeric_col = tf.keras.Input(shape=(1,), name='distance')
hour_col = tf.keras.Input(shape=(1,), name='hour', dtype='int64')
weekday_col = tf.keras.Input(shape=(1,), name='weekday', dtype='int64')
```

接著建立正規化和類別編碼層，並將這些層儲存在清單中。

```

all_inputs = []
encoded_features = []

# Pass 'distance' input to normalization layer
normalization_layer = get_normalization_layer('distance', train_dataset)
encoded_numeric_col = normalization_layer(numeric_col)
all_inputs.append(numeric_col)
encoded_features.append(encoded_numeric_col)

# Pass 'hour' input to category encoding layer
encoding_layer = get_category_encoding_layer('hour', train_dataset, dtype='int64')
encoded_hour_col = encoding_layer(hour_col)
all_inputs.append(hour_col)
encoded_features.append(encoded_hour_col)

# Pass 'weekday' input to category encoding layer
encoding_layer = get_category_encoding_layer('weekday', train_dataset, dtype='int64')
encoded_weekday_col = encoding_layer(weekday_col)
all_inputs.append(weekday_col)
encoded_features.append(encoded_weekday_col)

```

定義前處理層後，即可定義模型的其餘部分。您將串連所有輸入特徵，並傳遞至稠密層。由於這是迴歸問題，因此輸出層是單一單位。

```

all_features = tf.keras.layers.concatenate(encoded_features)
x = tf.keras.layers.Dense(64, activation="relu")(all_features)
output = tf.keras.layers.Dense(1)(x)
model = tf.keras.Model(all_inputs, output)

```

最後，編譯模型。

```

model.compile(optimizer = tf.keras.optimizers.Adam(0.001),
              loss='mean_squared_logarithmic_error')

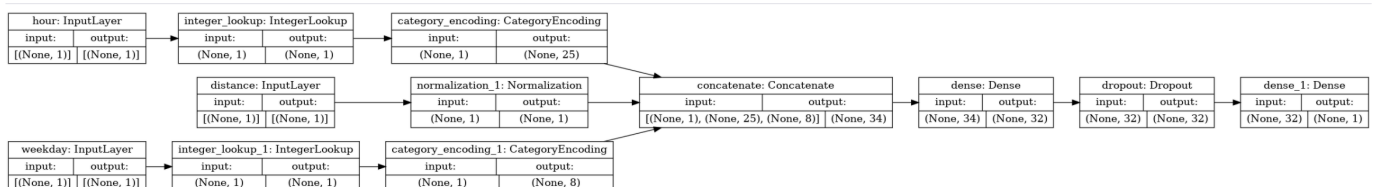
```

定義模型後，您就可以將架構視覺化

```

tf.keras.utils.plot_model(model, show_shapes=True, rankdir="LR")

```



請注意，這個模型對這個簡單的資料集來說相當複雜。僅供示範之用。

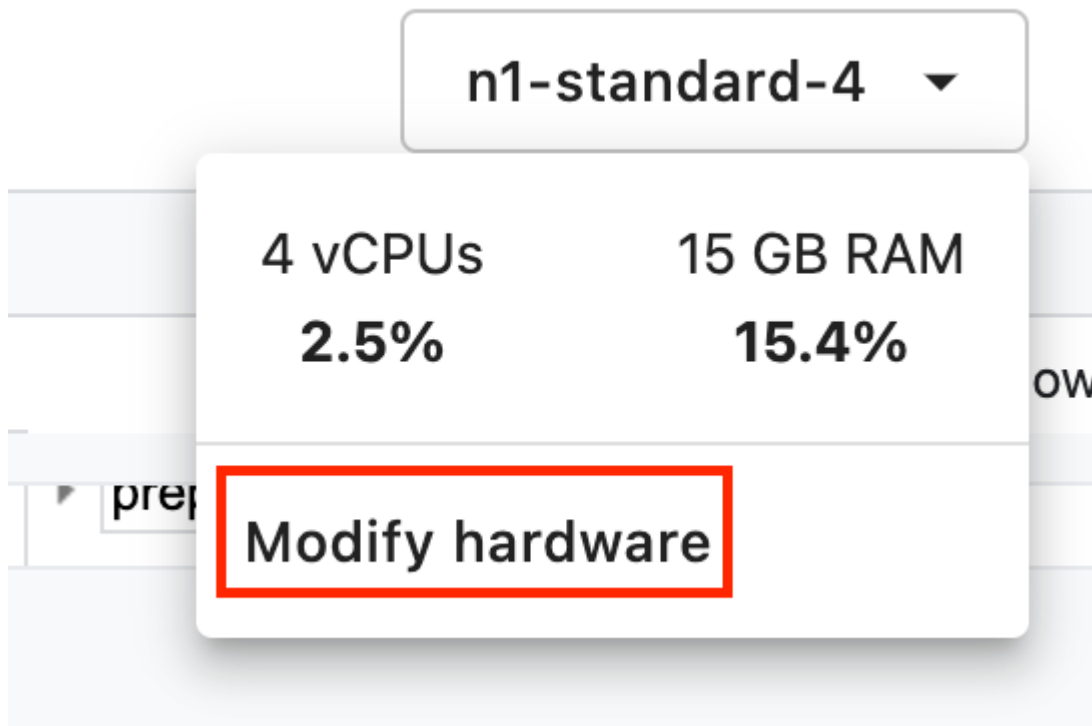
我們訓練 1 個週期，確認程式碼可以執行。

```
model.fit(train_dataset, validation_data = validation_dataset, epochs = 1)
```

使用 GPU 訓練模型

接著，您會訓練模型更久的時間，並使用硬體切換器加快訓練速度。使用 Vertex AI Workbench 時，您可以在不關閉執行個體的情況下變更硬體。只在需要時新增 GPU，有助於降低成本。

如要變更硬體設定檔，請按一下右上角的機型，然後選取「修改硬體」



選取「Attach GPUs」(附加 GPU)，然後選取 NVIDIA T4 Tensor Core GPU。

Modify Hardware Configuration

Modify the hardware configuration of your virtual machine as you need. The options available for the hardware are configured by your administrators. [Learn More](#) 

Machine Configuration

Machine type

n1-standard-4 (4 vCPUs, 15 GB RAM) ▼

GPU Configuration

Based on the zone, framework, and machine type of the instance, the available GPU types and the minimum number of GPUs that can be selected may vary. [Learn More](#) 

☒ Attach GPUs

GPU type

NVIDIA Tesla T4 ▼

Number of GPUs

1 ▼



If you have chosen to attach GPUs to your instance, the NVIDIA GPU driver will be installed automatically on the next startup.

Cancel

Next

硬體設定大約需要五分鐘。程序完成後，請再訓練模型一段時間。你會發現現在每個訓練週期所需的時間較少。

```
model.fit(train_dataset, validation_data = validation_dataset, epochs = 5)
```

你認為有哪些方法可以提高預測準確度？不妨試著將週期資料與美國國家海洋暨大氣總署 (NOAA) 的[每日天氣資料](#)合併。

🎉 恭喜！🎉

您已學會如何使用 Vertex AI Workbench 執行下列作業：

- 探索 BigQuery 中的資料
- 使用 BigQuery 用戶端將資料載入 Python
- 使用 Keras 前處理層和 GPU 訓練 TensorFlow 模型

如要進一步瞭解 Vertex AI 的其他部分，請參閱[說明文件](#)。

7. 清除所用資源

由於我們將筆記本設定為在閒置 60 分鐘後逾時，因此不必擔心執行個體會關閉。如要手動關閉執行個體，請按一下控制台 Vertex AI Workbench 專區的「停止」按鈕。如要徹底刪除筆記本，請按一下「刪除」按鈕。

